

Aufgabenstellung: Interfaces in Java

(Interfaces „IGitarrenspieler“, „ISänger“ sowie Klasse „Musiker“ und Arbeiten mit mehrfach implementierten Interfaces und UML-Klassendiagramm)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das zwei Interfaces und eine konkrete Klasse verwendet.

Sie lernen dabei,

- was ein Interface in Java ist,
- wie eine Klasse mehrere Interfaces gleichzeitig implementieren kann,
- wie Methoden aus Interfaces in einer Klasse konkret umgesetzt werden,
- wie gemeinsame Fähigkeiten durch Interfaces beschrieben werden,
- wie ein Testprogramm die Implementierung der Interfaces sichtbar macht,
- wie die Struktur eines Programms in einem UML-Klassendiagramm dargestellt werden kann.

Der Fokus liegt auf dem Verständnis, dass Interfaces in Java

Fähigkeiten oder Rollen beschreiben, die von konkreten Klassen umgesetzt werden müssen. Eine Klasse kann dabei mehrere Interfaces gleichzeitig implementieren und damit mehrere Fähigkeiten in sich vereinen.

Fachlicher Hintergrund

In der objektorientierten Programmierung werden nicht nur Klassen und Vererbung verwendet, sondern auch Interfaces.

Ein Interface beschreibt, **welche Methoden** eine Klasse bereitstellen muss, ohne bereits festzulegen, **wie** diese Methoden konkret umgesetzt werden.

In dieser Aufgabe werden zwei Fähigkeiten modelliert:

- Gitarre spielen
- singen

Eine konkrete Klasse kann beide Fähigkeiten besitzen. Dadurch eignet sich dieses Beispiel besonders gut, um zu zeigen, dass eine Klasse in Java mehrere Interfaces gleichzeitig implementieren kann.

Typische Beziehungen in dieser Aufgabe sind:

- ein Musiker ist Gitarrenspieler,
- ein Musiker ist Sänger,

- ein Musiker vereint beide Fähigkeiten in einer Klasse.

Damit eignet sich dieses Beispiel gut, um die Mehrfachimplementierung von Interfaces in Java grundlegend zu verstehen.

Vorgegebene Klassen- und Interfacestruktur

Im Projekt sollen folgende Interfaces und Klassen entwickelt und verwendet werden:

- `IGitarrenspieler`
- `ISänger`
- `Musiker`
- `Main`

Die Bestandteile haben dabei folgende Rollen:

- `IGitarrenspieler` beschreibt die Fähigkeit, Gitarre zu spielen,
- `ISänger` beschreibt die Fähigkeit, zu singen,
- `Musiker` ist eine konkrete Klasse, die beide Interfaces implementiert,
- `Main` enthält die `main`-Methode zum Testen des Programms.

Zusätzlich soll ein **UML-Klassendiagramm** erstellt werden, das die Beziehungen zwischen den Interfaces und der Klasse darstellt.

Aufgabe 1: Entwicklung des Interfaces „IGitarrenspieler“

Ziel

Sie entwickeln das Interface `IGitarrenspieler` als allgemeine Beschreibung für Objekte, die Gitarre spielen können.

Zu entwickelnde Struktur

Das Interface `IGitarrenspieler` soll mindestens enthalten:

```
void gitarreSpielen();
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface `IGitarrenspieler` so, dass:

- die Fähigkeit des Gitarrespielens beschrieben wird,
- eine passende Methode zum Gitarrespielen vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `interface`,

- mindestens eine Methodendeklaration mit dem Namen `gitarreSpielen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was ein Interface in Java ist,
 - wie eine Fähigkeit unabhängig von einer konkreten Klasse beschrieben wird,
 - dass ein Interface nur eine Vorgabe für Methoden enthält.
-

Aufgabe 2: Entwicklung des Interfaces „ISänger“

Ziel

Sie entwickeln das Interface `ISänger` als allgemeine Beschreibung für Objekte, die singen können.

Zu entwickelnde Struktur

Das Interface `ISänger` soll mindestens enthalten:

```
void singen();
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface `ISänger` so, dass:

- die Fähigkeit des Singens beschrieben wird,
- eine passende Methode zum Singen vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `interface`,
- mindestens eine Methodendeklaration mit dem Namen `singen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass unterschiedliche Fähigkeiten in getrennten Interfaces beschrieben werden können,
- wie mehrere Rollen unabhängig voneinander modelliert werden.

Aufgabe 3: Entwicklung der Klasse „Musiker“

Ziel

Sie entwickeln die Klasse `Musiker` als konkrete Klasse, die beide Interfaces implementiert.

Zu entwickelnde Klassenstruktur

```
public class Musiker implements IGitarrenspieler,  
ISänger
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Musiker` so, dass:

- sie die Fähigkeit besitzt, Gitarre zu spielen,
- sie die Fähigkeit besitzt, zu singen,
- beide Methoden bei ihrem Aufruf eine sinnvolle Konsolenausgabe erzeugen.

Technische Anforderungen

Verwenden Sie:

- `implements IGitarrenspieler, ISänger,`
- eine konkrete Implementierung der Methode `gitarreSpielen()`,
- eine konkrete Implementierung der Methode `singen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine Klasse mehrere Interfaces gleichzeitig implementiert,

- dass alle im Interface vorgegebenen Methoden konkret umgesetzt werden müssen,
 - wie mehrere Fähigkeiten in einer einzigen Klasse zusammengeführt werden können.
-

Aufgabe 4: Analyse der mehrfachen Interface-Implementierung

Ziel

Sie analysieren, warum die Klasse `Musiker` beide Interfaces implementiert.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- warum `Musiker` sowohl `IGitarrenspieler` als auch `ISänger` implementiert,
- welche Vorteile es hat, Fähigkeiten in getrennten Interfaces zu beschreiben,
- warum eine Klasse mehrere Interfaces gleichzeitig umsetzen kann.

Technische Anforderungen

Beziehen Sie sich auf:

- `implements`,
- mehrere Interfaces in einer Klasse,
- gemeinsame und unterschiedliche Fähigkeiten.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Mehrfachimplementierung mit Interfaces in Java funktioniert,
 - warum Interfaces eine flexible Modellierung von Fähigkeiten ermöglichen,
 - wie Klassen verschiedene Rollen gleichzeitig übernehmen können.
-

Aufgabe 5: Entwicklung der Testklasse „Main“

Ziel

Sie entwickeln ein Testprogramm, das die Implementierung der Interfaces praktisch demonstriert.

Zu entwickelnde Klasse

Die Klasse `Main` soll eine `main`-Methode enthalten.

Anforderungen

Funktionale Anforderungen

Das Testprogramm soll:

1. ein Objekt der Klasse `Musiker` erzeugen,
2. die Methode `gitarreSpielen()` aufrufen,
3. die Methode `singen()` aufrufen,
4. durch die Ausgabe in der Konsole sichtbar machen, dass `Musiker` beide Interfaces umgesetzt hat.

Technische Anforderungen

Verwenden Sie:

- eine `main`-Methode,
- eine Objekterzeugung der Klasse `Musiker`,
- Methodenaufrufe auf dem erzeugten Objekt,
- sinnvolle Konsolenausgaben.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie implementierte Interfaces in einem Testprogramm verwendet werden,
- wie die konkrete Umsetzung der Methoden sichtbar gemacht wird,

- wie eine Klasse mit mehreren Fähigkeiten praktisch getestet werden kann.
-

Aufgabe 6: Erstellung eines UML-Klassendiagramms

Ziel

Sie erstellen ein UML-Klassendiagramm zur dargestellten Programmstruktur.

Anforderungen

Funktionale Anforderungen

Erstellen Sie ein UML-Klassendiagramm, in dem sichtbar wird:

- dass `IGitarrenspieler` ein Interface ist,
- dass `ISänger` ein Interface ist,
- dass `Musiker` eine konkrete Klasse ist,
- dass `Musiker` beide Interfaces implementiert,
- welche Methoden in den Interfaces und in der Klasse eine Rolle spielen.

Technische Anforderungen

Stellen Sie im Diagramm dar:

- die beiden Interfaces,
- die Klasse `Musiker`,
- die Implementierungsbeziehungen,
- die Methodennamen `gitarreSpielen()` und `singen()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Programmstrukturen grafisch dargestellt werden,
- wie Interfaces und Klassen im UML-Klassendiagramm voneinander unterschieden werden,
- wie Implementierungsbeziehungen in UML sichtbar gemacht werden.

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was ein Interface in Java ist,
- den Unterschied zwischen Interface und konkreter Klasse beschreiben,
- eine Klasse mit `implements` modellieren,
- mehrere Interfaces in einer Klasse gleichzeitig implementieren,
- Methoden aus Interfaces konkret umsetzen,

- ein Testprogramm zur Überprüfung der Implementierung schreiben,
 - ein einfaches UML-Klassendiagramm für Interfaces und Klassen erstellen.
-

Didaktische Einordnung

Diese Aufgabe erweitert grundlegende Java-Kenntnisse um das Konzept der Interfaces.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Methoden
- Interfaces
- `implements`
- objektorientierte Modellierung
- UML-Klassendiagramme

Die Aufgabe zeigt in einer bewusst einfachen Struktur, dass Java-Klassen mehrere Fähigkeiten gleichzeitig besitzen können, wenn sie mehrere Interfaces implementieren.

Sie bildet eine wichtige Grundlage für weiterführende Themen wie:

- polymorphe Programmierung

- Schnittstellenorientierung
 - Mehrfachimplementierung
 - abstrakte Klassen
 - UML-Modellierung
 - objektorientierter Softwareentwurf
-

Reflexions- und Kontrollfragen

Kontrollfragen zu Interfaces

1. Was ist ein Interface in Java?
 2. Welche Aufgabe hat ein Interface?
 3. Warum beschreibt ein Interface eher eine Fähigkeit als eine konkrete Klasse?
 4. Warum kann eine Klasse mehrere Interfaces gleichzeitig implementieren?
-

Kontrollfragen zur Klassenstruktur

5. Warum implementiert `Musiker` sowohl `IGitarrenspieler` als auch `ISänger`?
 6. Welche Methoden muss `Musiker` konkret umsetzen?
 7. Welche Fähigkeiten werden durch die beiden Interfaces beschrieben?
 8. Warum ist `Musiker` eine konkrete Klasse und kein Interface?
-

Kontrollfragen zur Programmlogik

9. Was bedeutet `implements` in Java?
 10. Warum müssen die Methoden aus einem Interface in der Klasse implementiert werden?
 11. Wie zeigt das Testprogramm, dass `Musiker` beide Fähigkeiten besitzt?
 12. Welche Vorteile hat diese Modellierung für Erweiterbarkeit und Übersichtlichkeit?
-

Kontrollfragen zur UML-Darstellung

13. Wie werden Interfaces in einem UML-Klassendiagramm dargestellt?
 14. Wie wird die Beziehung zwischen `Musiker` und den Interfaces im Diagramm sichtbar?
 15. Warum ist ein UML-Klassendiagramm bei kleinen Programmen trotzdem hilfreich?
-